

Window Functions Cheat Sheet

Complete quick reference for all PostgreSQL window functions with syntax, frame options, and runnable examples.

Syntax Reference

```
function_name([arguments])
  OVER (
    [PARTITION BY partition_expression, ...]
    [ORDER BY sort_expression [ASC | DESC] [NULLS {FIRST | LAST}], ...]
    [frame_clause]
  )
```

Frame Clause Syntax

```
{ROWS | RANGE | GROUPS}
BETWEEN frame_start AND frame_end
```

Frame Start/End Options

Option	Meaning
UNBOUNDED PRECEDING	First row of partition
n PRECEDING	n rows/range units before current
CURRENT ROW	Current row (or current row's value range)
n FOLLOWING	n rows/range units after current
UNBOUNDED FOLLOWING	Last row of partition

Frame Mode Differences

Mode	Unit	Uses
ROWS	Physical rows	Moving averages, running totals
RANGE	Logical value range (ORDER BY value)	Group rows with same ORDER BY value
GROUPS	Groups of equal ORDER BY rows	Less common

GOTCHA: Default frame (when ORDER BY present): `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. This includes all rows with equal ORDER BY value as "current row." Use `ROWS` for predictable physical-row behavior.

All Window Functions — Reference Table

Ranking Functions

Function	Ties	Gaps	Result Example
ROW_NUMBER()	Unique	N/A	1, 2, 3, 4, 5
RANK()	Same rank	Yes	1, 1, 3, 4, 4
DENSE_RANK()	Same rank	No	1, 1, 2, 3, 3
NTILE(n)	N/A	N/A	1,1,2,2,3 (n=3)
PERCENT_RANK()	(rank-1)/(N-1)	N/A	0.0 to 1.0
CUME_DIST()	Cumulative	N/A	> 0 to 1.0

```
-- All ranking functions together
SELECT
    name, score,
    ROW_NUMBER() OVER (ORDER BY score DESC) AS row_num,
    RANK() OVER (ORDER BY score DESC) AS rnk,
    DENSE_RANK() OVER (ORDER BY score DESC) AS dense_rnk,
    NTILE(4) OVER (ORDER BY score DESC) AS quartile,
    PERCENT_RANK() OVER (ORDER BY score DESC) AS pct_rank,
    CUME_DIST() OVER (ORDER BY score DESC) AS cume_dist
FROM student_scores;
```

Value / Offset Functions

Function	Description
LAG(col, [offset=1], [default])	Value from N rows before current
LEAD(col, [offset=1], [default])	Value from N rows after current
FIRST_VALUE(col)	First value in window frame
LAST_VALUE(col)	Last value in window frame
NTH_VALUE(col, n)	Nth value in window frame

```
-- LAG: compare to previous row
SELECT
    date, sales,
    LAG(sales) OVER (ORDER BY date) AS prev_sales,
    LAG(sales, 2, 0) OVER (ORDER BY date) AS two_periods_ago, -- default 0 if no
row
    sales - LAG(sales) OVER (ORDER BY date) AS day_over_day_change
FROM daily_sales;

-- LEAD: look ahead
SELECT
```

```

    order_id, order_date,
    LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS
next_order_date,
    LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) -
order_date AS days_to_next_order
FROM orders;

-- FIRST_VALUE / LAST_VALUE
SELECT
    employee_id, department_id, salary,
    FIRST_VALUE(salary) OVER (PARTITION BY department_id ORDER BY salary DESC) AS
max_salary_in_dept,
    LAST_VALUE(salary) OVER (
        PARTITION BY department_id ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING -- MUST specify
full frame!
    ) AS min_salary_in_dept
FROM employees;

```

GOTCHA: `LAST_VALUE` with default frame only sees current row to partition end. ALWAYS specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` for `LAST_VALUE` to work correctly across partition.

```

-- NTH_VALUE: get 2nd highest salary per department
SELECT
    department_id, employee_id, salary,
    NTH_VALUE(salary, 2) OVER (
        PARTITION BY department_id
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS second_highest_in_dept
FROM employees;

```

Aggregate Window Functions

All standard aggregate functions work as window functions: `SUM`, `COUNT`, `AVG`, `MIN`, `MAX`, `STDDEV`, `VARIANCE`, `STRING_AGG`, `ARRAY_AGG`, `BOOL_AND`, `BOOL_OR`

```

-- Running total (cumulative sum)
SELECT
    date, amount,
    SUM(amount) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING) AS cumulative_total
FROM daily_revenue;

-- Percentage of partition total
SELECT
    department_id, employee_id, salary,
    SUM(salary) OVER (PARTITION BY department_id) AS dept_total,
    ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY department_id), 2) AS
pct_of_dept,

```

```

        ROUND(100.0 * salary / SUM(salary) OVER (), 2) AS pct_of_company
FROM employees;

-- Moving average (7-day)
SELECT
    date, value,
    AVG(value) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
ma_7d,
    AVG(value) OVER (ORDER BY date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW) AS
ma_30d
FROM daily_metrics;

-- Min/Max within rolling window
SELECT
    date, price,
    MAX(price) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
high_7d,
    MIN(price) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
low_7d
FROM stock_prices;

-- Count within partition
SELECT
    department_id, employee_id,
    COUNT(*) OVER (PARTITION BY department_id) AS dept_size
FROM employees;

-- Running count of events per user
SELECT
    user_id, event_time, event_type,
    COUNT(*) OVER (PARTITION BY user_id ORDER BY event_time ROWS UNBOUNDED
PRECEDING) AS event_num
FROM user_events;

```

Common Patterns

Pattern 1: Top N Per Group

```

-- Top 3 products by sales per category
SELECT category_id, product_id, sales
FROM (
    SELECT
        category_id, product_id, sales,
        RANK() OVER (PARTITION BY category_id ORDER BY sales DESC) AS rnk
    FROM product_sales
) ranked
WHERE rnk <= 3;

-- Latest N rows per entity (DISTINCT ON is often faster for N=1)
SELECT DISTINCT ON (customer_id) customer_id, order_id, order_date, amount

```

```
FROM orders
ORDER BY customer_id, order_date DESC;
```

Pattern 2: Period-over-Period Comparison

```
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) AS absolute_change,
    ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
        / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS pct_change,
    -- Compare same month last year
    LAG(revenue, 12) OVER (ORDER BY month) AS same_month_last_year
FROM monthly_revenue;
```

Pattern 3: Running Total with Reset (e.g., per year)

```
SELECT
    year, month, monthly_sales,
    SUM(monthly_sales) OVER (
        PARTITION BY year
        ORDER BY month
        ROWS UNBOUNDED PRECEDING
    ) AS ytd_sales -- resets each year
FROM monthly_sales;
```

Pattern 4: Gaps and Islands

```
-- Identify consecutive date ranges (islands)
WITH numbered AS (
    SELECT
        date_col,
        date_col - (ROW_NUMBER() OVER (ORDER BY date_col))::int AS island_key
    FROM active_dates
)
SELECT
    MIN(date_col) AS island_start,
    MAX(date_col) AS island_end,
    MAX(date_col) - MIN(date_col) + 1 AS duration_days
FROM numbered
GROUP BY island_key
ORDER BY island_start;
```

Pattern 5: Session Analysis

```

-- Group events into sessions (30-min gap = new session)
WITH with_gap AS (
    SELECT
        user_id, event_time,
        LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) AS prev_time
    FROM events
),
sessions AS (
    SELECT
        user_id, event_time,
        SUM(CASE WHEN prev_time IS NULL OR event_time - prev_time > INTERVAL '30
min'
            THEN 1 ELSE 0 END)
            OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
    FROM with_gap
)
SELECT user_id, session_id, MIN(event_time) AS start, MAX(event_time) AS end,
    COUNT(*) AS events, MAX(event_time) - MIN(event_time) AS duration
FROM sessions
GROUP BY user_id, session_id;

```

Pattern 6: Percentile / Distribution

```

SELECT
    category,
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY price) AS q1,
    PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY price) AS median,
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY price) AS q3,
    PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY price) AS p95,
    AVG(price) AS mean
FROM products
GROUP BY category;

```

Note: `PERCENTILE_CONT` is an ordered-set aggregate (not window function), but commonly tested alongside window functions.

Pattern 7: Cohort Retention

```

WITH cohorts AS (
    SELECT user_id, DATE_TRUNC('month', MIN(created_at)) AS cohort_month
    FROM users GROUP BY user_id
),
user_activity AS (
    SELECT DISTINCT user_id, DATE_TRUNC('month', occurred_at) AS active_month
    FROM events
)
SELECT
    c.cohort_month,
    (EXTRACT(YEAR FROM ua.active_month) - EXTRACT(YEAR FROM c.cohort_month)) * 12 +

```

```

        (EXTRACT(MONTH FROM ua.active_month) - EXTRACT(MONTH FROM c.cohort_month)) AS
month_number,
        COUNT(DISTINCT ua.user_id) AS retained
FROM cohorts c
JOIN user_activity ua ON c.user_id = ua.user_id AND ua.active_month >=
c.cohort_month
GROUP BY c.cohort_month, month_number
ORDER BY c.cohort_month, month_number;

```

Pattern 8: Deduplication (Keep Latest)

```

-- Keep only the most recent record per entity
DELETE FROM staging_data
WHERE id NOT IN (
    SELECT id FROM (
        SELECT id, ROW_NUMBER() OVER (PARTITION BY entity_id ORDER BY updated_at
DESC) AS rn
        FROM staging_data
    ) ranked
    WHERE rn = 1
);

```

Pattern 9: Conditional Running Total

```

-- Running total only for positive values
SELECT
    date, amount,
    SUM(CASE WHEN amount > 0 THEN amount ELSE 0 END)
    OVER (ORDER BY date ROWS UNBOUNDED PRECEDING) AS running_positive_total
FROM transactions;

```

Pattern 10: Sparse Data Fill (forward fill / LOCF)

```

-- Last-observation-carried-forward for NULLs
SELECT
    date,
    value,
    FIRST_VALUE(value) OVER (
        ORDER BY date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS value_ffill -- only works if first row is not null
FROM time_series_with_gaps;

-- Proper LOCF using a subquery
SELECT
    date,
    MAX(value) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING) AS filled_value
FROM (

```

```
SELECT date, value FROM time_series -- NULLs present
) t;
-- Note: MAX ignores NULLs, so MAX over all preceding = last non-null value
```

WINDOW Clause (Named Windows)

```
-- Define window once, reference multiple times
SELECT
  employee_id, salary, department_id,
  AVG(salary)      OVER dept_window AS dept_avg,
  MAX(salary)      OVER dept_window AS dept_max,
  RANK()           OVER dept_window AS dept_rank,
  SUM(salary)      OVER company_window AS company_total
FROM employees
WINDOW
  dept_window      AS (PARTITION BY department_id ORDER BY salary DESC),
  company_window   AS ();
```

Performance Tips

Scenario	Recommendation
Multiple windows on same partition	Use WINDOW clause to avoid recalculation
COUNT(DISTINCT) in window	Not supported natively — use subquery or HLL extension
Avoid window on unsorted large table	Ensure ORDER BY column has an index
Frame RANGE VS ROWS	ROWS is faster and more predictable
Complex window computation	Consider materializing intermediate results
Multiple different windows	PostgreSQL may sort multiple times — check EXPLAIN

```
-- EXPLAIN to see Window node costs
EXPLAIN (ANALYZE, BUFFERS) SELECT
  user_id,
  SUM(amount) OVER (PARTITION BY user_id ORDER BY date ROWS UNBOUNDED PRECEDING)
FROM transactions;
-- Look for: "WindowAgg" node with Sort underneath
```

Quick Cheat Table

Task	Function + Frame
Row number (no ties)	ROW_NUMBER() OVER (ORDER BY col)

Rank with gaps	<code>RANK() OVER (ORDER BY col DESC)</code>
Rank without gaps	<code>DENSE_RANK() OVER (ORDER BY col DESC)</code>
Bucket into N groups	<code>NTILE(N) OVER (ORDER BY col)</code>
Running sum	<code>SUM(col) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)</code>
Running average	<code>AVG(col) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)</code>
7-day moving avg	<code>AVG(col) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)</code>
Previous row	<code>LAG(col) OVER (PARTITION BY x ORDER BY date)</code>
Next row	<code>LEAD(col) OVER (PARTITION BY x ORDER BY date)</code>
First in partition	<code>FIRST_VALUE(col) OVER (PARTITION BY x ORDER BY date)</code>
Last in partition	<code>LAST_VALUE(col) OVER (PARTITION BY x ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)</code>
% of total	<code>col / SUM(col) OVER () * 100</code>
% of group total	<code>col / SUM(col) OVER (PARTITION BY group) * 100</code>
Period-over-period %	<code>(col - LAG(col) OVER ()) / NULLIF(LAG(col) OVER (), 0) * 100</code>
Cumulative distribution	<code>CUME_DIST() OVER (ORDER BY col)</code>
Percentile rank	<code>PERCENT_RANK() OVER (ORDER BY col)</code>